

Report Exercise 1
Advanced Computer Architectures

()

Michiel Vandromme & Joffrey Van Cauwenberghe

February 17, 2024



Class: Master Software Engineering

Contents

1 Introduction **2**

2 Implementation **2**

3 Conclusion **2**

1 Introduction

In this lab, we delved into the realm of GPU programming utilizing the CUDA interface within C++. Our primary objective was to harness the immense parallel execution capabilities of the GPU to efficiently flip a large array of integers.

2 Implementation

Since the CPU and GPU have separate memory spaces, we need to copy the data from the CPU or host memory to the GPU, also known as the device memory.
First and foremost, we had to construct our input array in the host memory in the traditional C++ manner. Subsequently, we allocated some device memory for the input and output arrays using the CudaMemAlloc method. The CudaMemcpy method allowed us to copy the input array to this allocated memory.
Afterwards, we only need to configure our GPU by defining the number of threads per block and the number of blocks. We opted for 256 threads per block. This number is free to choose, but after some research, we found that many hardware architectures were optimised for a multiple of 32 threads per block. The number of blocks is equal to the size of the array plus the number of threads per block divided by that number of threads per block. This ensures that all elements are covered.
Finally, only the flip_array method needs to be invoked, where we pass the GPU configuration and the allocated input and output arrays. The output is copied back to the host memory, allowing us to print the array.
In the method that flips the array, we first retrieve the thread index. Subsequently, we perform a check to verify whether this thread index is within the range of the array. If this is the case, we place the corresponding element in the correct position in the output array.

3 Conclusion

This first exercise introduced us to the operation of the GPU for the first time and how we ourselves can address the GPU from within our code. This provides a good basis for the rest of the lab exercises.